Risposte Esame di Tecnologie Web (Facsimile)

ID Prova: 00001

Esercizio 1 (2 punti)

Qual è il significato degli status code della famiglia 400 di HTTP?

Risposta: B. Errore nella richiesta.

I codici di stato HTTP della serie 400 indicano errori del client dove il problema è con la richiesta stessa (come 400 Bad Request, 401 Unauthorized, 404 Not Found).

Esercizio 2 (2 punti)

Quale dei seguenti selettori CSS seleziona tutti i link contenuti in `<li>` di posto pari in una lista non ordinata con classe lst?

Risposta: D. ul.lst li

(even) a

Questo seleziona correttamente: liste non ordinate con classe "lst", poi elementi di lista in posizione pari, poi elementi ancora all'interno di questi elementi.

Esercizio 3 (2 punti)

Qual è il colore finale dell'elemento h1 date le regole CSS e l'HTML?

Risposta: F. Pink

La regola `div h1 { color: pink !important; }` vince perché:

- Seleziona correttamente il nostro elemento h1
- Ha il flag !important che sovrascrive lo stile inline
- Le altre regole con !important non selezionano correttamente il nostro elemento (h1.f vs h1 con class="f")

Esercizio 4 (2 punti)

Con riferimento al ciclo di vita di una variabile dichiarata con let:

Risposte corrette: C e D

- C. In Linea 3, x è dichiarata ma non inizializzata (nella temporal dead zone)
- D. In Linea 5, x ha valore undefined (dopo la dichiarazione)

Il codice non esegue senza errori a causa di errori di riferimento, quindi E è falsa.

Esercizio 5 (2 punti)

Quali dei seguenti non sono tipi di dato primitivi in JavaScript?

Risposte corrette: A, C, E

- A. array (oggetto, non primitivo)
- C. Promise (oggetto, non primitivo)
- E. float (non è un tipo separato in JavaScript)

Esercizio 6 (2 punti)

Qual è il risultato dell'esecuzione del frammento try-catch?

Risposta: A. A, X, Z.

In modalità strict:

1. "A" viene registrato
2. `x = 1` genera un ReferenceError (x non dichiarato)
3. Il blocco catch viene eseguito, registrando "X"
4. Il blocco finally viene sempre eseguito, registrando "Z"

Esercizio 7 (2 punti)

Qual è il risultato dell'esecuzione del frammento Promise?

Risposta: D. Done, B, A.

1. "Done" viene registrato immediatamente (codice sincrono)
2. Dopo il timeout, la Promise si risolve
3. Il gestore finally viene eseguito per primo, registrando "B"
4. Poi il gestore then viene eseguito, registrando "A"

Esercizio 8 (2 punti)

Qual è l'output in console dopo il caricamento completo del documento HTML?

Risposta: E. `<meta charset="UTF-8"/>`.

L'head ha due figli: title (indice 0) e meta (indice 1). Il codice accede a h[1] che è l'elemento meta.

Esercizio 9 (5 punti)

Quali sono gli svantaggi dell'utilizzo di un approccio multi-threaded request handling per un'applicazione web che effettua frequenti accessi in lettura/scrittura a un database?

Risposta:
L'approccio multi-threaded request handling presenta diversi svantaggi per applicazioni con frequente accesso al database:

1. **Consumo di risorse**: Ogni thread richiede memoria e risorse CPU, che possono portare all'esaurimento delle risorse con molte richieste concorrenti.
2. **Limitazioni di connessione al database**: I database tipicamente hanno un numero massimo di connessioni concorrenti che possono essere rapidamente esaurite.
3. **Overhead di sincronizzazione**: Quando più thread accedono agli stessi dati, diventano necessari meccanismi di blocco complessi, aggiungendo overhead.
4. **Rischio di deadlock**: Più thread che accedono a risorse condivise aumentano la probabilità di deadlock, dove i thread attendono indefinitamente risorse detenute da altri.
5. **Sfide di scalabilità**: Il multi-threading non scala linearmente, specialmente per operazioni I/O-bound come l'accesso al database.

Approcci alternativi come architetture event-driven o connection pooling possono essere più efficienti per applicazioni intensive di database.

Esercizio 10 (5 punti)

Analogie e differenze tra gli attributi async e defer in un elemento <script>.

Risposta:
Entrambi gli attributi `async` e `defer` sono attributi per elementi `<script>` che migliorano il caricamento della pagina permettendo al parsing HTML di continuare mentre si gestiscono gli script.

Differenze principali:

1. **Tempistica di esecuzione**:
   - `async`: Esegue lo script non appena viene scaricato, potenzialmente interrompendo il parsing HTML
   - `defer`: Attende l'esecuzione fino a quando il documento HTML non è completamente analizzato
2. **Ordine di esecuzione**:
   - `async`: Gli script possono essere eseguiti in qualsiasi ordine quando diventano disponibili
   - `defer`: Gli script vengono eseguiti nell'ordine in cui appaiono nel documento
3. **Casi d'uso**:
   - `async`: Ideale per script indipendenti che non dipendono dal DOM o da altri script

- `defer`: Migliore per script che necessitano di accesso al DOM o dipendono da altri script in sequenza

Esercizio 11 (5 punti)

Quali sono le possibili origini di regole CSS nello step di analisi dei bucket di origine-importanza nell'algoritmo cascade di risoluzione dei conflitti?

Risposta:
L'algoritmo cascade di CSS considera queste origini (in ordine di precedenza crescente):

1. **Origine User Agent (UA)**: Stili predefiniti del browser
2. **Origine utente**: Stili definiti dall'utente (preferenze del browser)
3. **Origine autore**: Stili forniti dallo sviluppatore (esterni, incorporati, inline)

Ogni origine può avere regole contrassegnate con `!important`, creando tre bucket di importanza aggiuntivi:

4. **Origine autore con !important**
5. **Origine utente con !important**
6. **Origine User Agent con !important**

Questo ordinamento garantisce che gli stili dell'autore generalmente sovrascrivano i valori predefiniti del browser, consentendo agli utenti di sovrascrivere gli stili dell'autore con dichiarazioni !important quando necessario.